LEARN BEING FORWARD

Programming — JavaScript Programming

Asynchronous JS — promises & async/await

COMPLETE LEARNING GUIDE

What You'll Learn

JavaScript runs on a single thread, yet web pages constantly wait on things that take real time — a network request, a timer, a file read — without ever freezing the page, and understanding exactly how that's possible (the event loop, task queues) is what separates a developer who can write async code from one who can actually debug it when it misbehaves. This sub-module traces the evolution from raw callbacks through Promises to async/await, covering Promise states and chaining, the four Promise combinators and when each is the right tool, proper error handling with try/catch around awaited code, and the subtle but important distinction between timers and microtasks that explains why asynchronous code doesn't always run in the order a beginner expects.

1. Why JavaScript Is Single-Threaded: the Event Loop

JavaScript executes all synchronous code on one single call stack, one operation at a time — there's no true parallel execution of JS code within a single context. Asynchronous operations (timers, network requests, file reads) are handed off to the browser or Node.js runtime itself, which performs the actual waiting outside of JavaScript entirely and queues a callback to run once the result is ready. The event loop is the mechanism that continuously checks whether the call stack is empty, and if so, pulls the next queued callback off a task queue and runs it — which is exactly why asynchronous code never blocks the page from responding to other things in the meantime.

JAVASCRIPT

```
console.log("1: synchronous");
setTimeout(() => console.log("3: after the timer callback"), 0);
console.log("2: also synchronous");
// Output order: 1, 2, 3 — the timer callback waits for ALL synchronous code to finish first,
// even with a 0ms delay, because it's queued, not run immediately.
```

2. Callbacks and Callback Hell

The original asynchronous pattern in JavaScript was passing a callback function to be invoked once an operation finished. Chaining multiple dependent asynchronous steps this way nests callback inside callback inside callback — colloquially 'callback hell' — producing code that drifts rightward with every added step and makes consistent error handling awkward, since each nested callback needs its own separate error-handling branch rather than one shared path.

```javascript
getUser(id, (user) => {
  getOrders(user.id, (orders) => {
    getOrderDetails(orders[0].id, (details) => {
      console.log(details); // three levels deep, and growing with every new step
    });
  });
});
```

3. Promises: Pending, Fulfilled, Rejected

A Promise represents a value that will exist at some point in the future, always in exactly one of three states: pending (not yet settled), fulfilled (completed successfully, with a resulting value), or rejected (failed, with a reason). Once a Promise settles — fulfilled or rejected — it stays in that state permanently; it can never flip back to pending or switch between fulfilled and rejected. Promises are constructed with new Promise((resolve, reject) => {...}), though in practice most code consumes Promises returned by existing APIs (like fetch) rather than constructing new ones directly.

```javascript
function delay(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function demo() {
  console.log("waiting...");
  await delay(1000);
  console.log("done waiting"); // runs ~1 second later
}
demo();
```

4. Chaining with .then, .catch, and .finally

.then(onFulfilled) attaches a callback to run once a Promise fulfills, and itself returns a new Promise — which is what makes chaining multiple sequential async steps possible without nesting, as long as each .then() returns either a plain value or another Promise for the next .then() to receive. .catch(onRejected) handles a rejection anywhere earlier in the chain, and .finally() runs regardless of outcome, commonly used for cleanup like hiding a loading indicator.

```javascript
fetchUser(id)
  .then((user) => fetchOrders(user.id)) // returning a Promise keeps the chain flat
  .then((orders) => renderOrders(orders))
  .catch((err) => console.error("Failed to load orders:", err))
  .finally(() => hideLoadingSpinner());
```

5. Promise Combinators: all, allSettled, race, any

Promise.all([...]) runs multiple Promises concurrently and resolves with an array of all their results once every one succeeds — but rejects immediately as soon as any single one fails, discarding the others' results. Promise.allSettled([...]) never rejects; it always resolves with a {status, value|reason} object per input Promise, which is the right choice when you need every result regardless of individual failures. Promise.race([...]) settles as soon as the first Promise settles, whichever way. Promise.any([...]) resolves as soon as the first Promise fulfills, only rejecting if every single one rejects.

```javascript
const results = await Promise.allSettled([
  fetchUser(1),
  fetchUser(999), // suppose this one fails
]);

results.forEach((r) => {
  if (r.status === "fulfilled") console.log("Got:", r.value);
  else console.warn("Failed:", r.reason);
});
```

6. async/await Syntax

async/await is syntax built on top of Promises that lets asynchronous code read top-to-bottom like ordinary synchronous code. A function marked async always returns a Promise, even if its body returns a plain value directly (which gets automatically wrapped). Inside an async function, await pauses only that function's execution until the awaited Promise settles, without blocking the rest of the program or the browser's main thread — other code elsewhere keeps running normally while the await is pending.

```javascript
async function loadUserOrders(id) {
  const user = await fetchUser(id);
  const orders = await fetchOrders(user.id);
  return orders;
}
// loadUserOrders(1) still returns a Promise — call it with await or .then()
```

7. Error Handling in Async Code

Inside an async function, wrapping await calls in a standard try/catch block catches a rejected Promise exactly like a thrown synchronous error — this is the main practical advantage async/await has over chained .then()/.catch() once several sequential steps are involved. An async function that throws (or awaits a rejected Promise) without any surrounding try/catch produces a rejected Promise itself, which must be handled by whoever calls it; an uncaught rejection anywhere in the chain surfaces as an 'unhandled promise rejection' warning or error.

```javascript
async function loadUserOrders(id) {
  try {
    const user = await fetchUser(id);
    const orders = await fetchOrders(user.id);
    return orders;
  } catch (err) {
    console.error("Failed to load orders:", err.message);
    return []; // fallback instead of letting the rejection propagate unhandled
  }
}
```

8. Timers: setTimeout, setInterval, and Clearing Them

setTimeout(callback, delay) schedules a callback to run once after at least the given delay in milliseconds (the delay is a minimum, not a guarantee, since the callback still has to wait its turn in the queue behind any currently running synchronous code). setInterval(callback, delay) repeats the callback indefinitely at that interval until explicitly stopped. Both return an ID that clearTimeout(id) or clearInterval(id) uses to cancel the scheduled or repeating callback — forgetting to clear an interval that's no longer needed is a common source of lingering background work and memory leaks in long-running pages.

```javascript
const intervalId = setInterval(() => console.log("tick"), 1000);

setTimeout(() => {
  clearInterval(intervalId); // stop the repeating tick after 5 seconds
  console.log("stopped");
}, 5000);
```

Common Pitfalls

- Assuming setTimeout(fn, 0) runs immediately — it still waits for all currently running synchronous code to finish first, because it's queued, not executed inline.
- Forgetting to return a Promise from inside a .then() callback, breaking the chain so the next .then() runs before the inner asynchronous work actually finishes.
- Forgetting await before a Promise-returning call, ending up with the Promise object itself instead of its resolved value.
- Awaiting calls inside a loop sequentially when they're actually independent and could run concurrently with Promise.all for much better performance.
- Using Promise.all when one failing request shouldn't cancel the others — Promise.allSettled is almost always the better fit for that case.
- Leaving an async function's rejected Promise completely unhandled by neither try/catch-ing it nor attaching a .catch() where it's called.

- Forgetting to clearInterval() a repeating timer that's no longer needed, leaving it running indefinitely in the background.

Key Takeaways

- JavaScript runs on a single thread; the event loop lets async work happen without blocking by queuing callbacks to run once the call stack is clear.
- A Promise settles exactly once, into either fulfilled or rejected, and can never change state again afterward.
- Always return a Promise (or value) from inside a .then() to keep a chain correctly sequenced.
- Promise.all fails fast on the first rejection; Promise.allSettled always resolves with every individual result or failure.
- async/await is syntax sugar over Promises — an async function always returns a Promise, and await only pauses that function, not the whole program.
- Wrap awaited calls in try/catch to handle rejections the same way you'd handle any other thrown error.

Learn More

MDN — Using Promises
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises

MDN — async function
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function

javascript.info — Event loop: microtasks and macrotasks
https://javascript.info/event-loop